

ATARA

Sistema de Alerta Temprana y Atención
al Rendimiento Académico

División de Educación Básica — MEP Costa Rica

Documentación Técnica y Funcional

de los Módulos Desarrollados

Curso: EIF408 — Proyecto de Ingeniería de Software

Fecha: Abril 2026

Versión API: 0.0.1-SNAPSHOT

Stack: Spring Boot 4.0.3 · Java 17 · PostgreSQL 15

Puerto: localhost:8081

Módulos cubiertos en este documento

1. Autenticación y Seguridad (JWT)
2. Control de Acceso por Rol (*ContextoUsuario*)
3. Evaluaciones por Saber
4. Alertas Temáticas
5. Catálogos de Saberes
6. Secciones y Docentes
7. Extracción PIAD (OCR)

Índice

1. Descripción General del Sistema	2
1.1. Stack tecnológico	2
1.2. Arquitectura en capas	2
1.3. Esquema de base de datos	2
2. Módulo 1: Autenticación y Seguridad (JWT)	4
2.1. Descripción funcional	4
2.2. Endpoints	4
2.3. Contrato de la petición de login	4
2.4. Reglas de seguridad	5
2.5. Roles disponibles	5
2.6. Evidencia de pruebas funcionales	5
3. Módulo 2: Control de Acceso por Rol (ContextoUsuario)	6
3.1. Descripción funcional	6
3.2. Estructura del ContextoUsuario	6
3.3. Flujo de construcción del contexto	6
3.4. Filtrado de catálogos por rol	6
3.5. Evidencia de pruebas funcionales	7
4. Módulo 3: Evaluaciones por Saber	8
4.1. Descripción funcional	8
4.2. Endpoints	9
4.3. Contrato de la petición	10
4.4. Escala de valoración por saber	11
4.5. Reglas de negocio	11
4.6. Ejes temáticos por materia	11
4.7. Evidencia de pruebas funcionales	11
5. Módulo 4: Alertas Temáticas	13
5.1. Descripción funcional	13
5.2. Endpoints	14
5.3. Respuesta de alerta	14
5.4. Reglas de generación de alertas	15

5.5. Evidencia de pruebas funcionales	16
6. Módulo 5: Catálogos de Saberes	17
6.1. Descripción funcional	17
6.2. Endpoints	17
6.3. Datos de referencia	17
7. Módulo 6: Secciones y Docentes	19
7.1. Descripción funcional	19
7.2. Endpoints	19
7.3. Datos de control de acceso para secciones	19
8. Módulo 7: Extracción PIAD (OCR)	20
8.1. Descripción funcional	20
8.2. Endpoint	20
8.3. Tecnología de procesamiento	20
8.4. Configuración requerida	20
9. Infraestructura de Pruebas Funcionales	21
9.1. Descripción	21
9.2. Arquitectura de las pruebas	21
9.3. Características técnicas	21
9.4. Resumen de resultados	21
10. Mapa completo de endpoints de la API	23
11. Conclusiones	24

Descripción General del Sistema

ATARA es una API REST desarrollada para la División de Educación Básica del Ministerio de Educación Pública (MEP) de Costa Rica. Su propósito es detectar de forma temprana dificultades académicas en los estudiantes de educación primaria, utilizando una escala de evaluación basada en **saberes** (Conceptual, Procedimental, Actitudinal) y generando **alertas temáticas** automáticas cuando el promedio de un eje temático cae por debajo del umbral de desempeño esperado.

Stack tecnológico

Componente	Tecnología	Versión
Framework	Spring Boot	4.0.3
Lenguaje	Java	17 (LTS)
Base de datos	PostgreSQL	15
Migraciones	Flyway	Integrado con Spring Boot
Seguridad	Spring Security + JWT (jjwt)	0.12.6
Validación	Jakarta Bean Validation	Integrado
OCR	Apache PDFBox + Tess4j	3.0.3 / 5.11.0
Pruebas	JUnit 5 + Testcontainers	1.20.4

Arquitectura en capas

Controller	->	Service (interface + impl)	->	Repository	->	Modelo JPA
@Valid DTOs		Reglas negocio ContextoUsuario		Spring Data JPA (JpaRepository)		

- **Controller:** recibe requests HTTP, delega al servicio y retorna DTOs.
- **Service:** contiene toda la lógica de negocio y validaciones. Nunca expone entidades JPA.
- **Repository:** extiende JpaRepository; solo consultas a la BD.
- **Modelo:** entidades JPA con FetchType.LAZY en todas las relaciones.
- **Seguridad:** JwtAuthenticationFilter valida el token en cada request. ContextoUsuarioService construye el ContextoUsuario del usuario autenticado.

Esquema de base de datos

El esquema es gestionado exclusivamente por **Flyway** (ddl-auto=none). Las migraciones ejecutadas son:

Migración	Contenido
V1	20 tablas base, 40+ índices, 13 triggers de auditoría
V2	Datos semilla: usuarios, centros, secciones, estudiantes, matrículas
V3	8 vistas de consulta (<code>vw_criterios_completos</code> , etc.)
V4	Módulo de saberes: <code>tipos_saber</code> , <code>ejes_tematicos</code> , <code>evaluaciones_saber</code> , <code>alertas_tematicas</code>
V5	Estudiantes adicionales
V6	Soporte multi-materia: <code>materia_id</code> en evaluaciones y alertas; 63 ejes para Matemáticas, Ciencias, Estudios Sociales
V7	Restricción única en <code>alertas_tematicas</code> por materia
V8	Tabla <code>usuario_materias</code> : asignación de materias por docente
V9	Usuario docente de prueba (Keylor Cortés)
V10	Asignación de materias adicionales para docente avargas
V11	Corrección de datos de sección

Módulo 1: Autenticación y Seguridad (JWT)

Descripción funcional

Este módulo gestiona la autenticación de usuarios mediante **JSON Web Tokens (JWT)**. Implementa un flujo de *access token* de corta duración con rotación de *refresh token*, y protege todos los endpoints de la API excepto los públicos.

Endpoints

Método	Ruta	Descripción	Auth
POST	/api/auth/login	Autentica usuario; retorna access token + refresh token	No
POST	/api/auth/refresh	Renueva el access token (rotación de refresh token)	No
POST	/api/auth/logout	Revoca el refresh token activo	No
GET	/api/auth/me	Retorna datos del usuario autenticado	Sí

Contrato de la petición de login

Listing 1: POST /api/auth/login — Request body

```
{
  "correo": "admin@atara.mep.go.cr",    // @NotBlank @Email
  "password": "Admin1234!"              // @NotBlank
}
```

Listing 2: POST /api/auth/login — Response body (200 OK)

```
{
  "accessToken": "<JWT>",
  "refreshToken": "<UUID>",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "userId": 1,
  "correo": "admin@atara.mep.go.cr",
  "nombre": "Administrador",
  "apellidos": "Sistema",
  "rol": "ADMIN"
}
```

Reglas de seguridad

- Las contraseñas se almacenan con **BCryptPasswordEncoder** (strength 12). Nunca se exponen en texto plano.
- El *access token* es validado en **JwtAuthenticationFilter** por cada request; nunca en controllers o services.
- El *refresh token* se rota en cada uso: el token anterior se revoca y se emite uno nuevo.
- Todos los endpoints requieren el header `Authorization: Bearer <token>`, excepto `/api/auth/login`, `/api/auth/refresh`, `/api/auth/logout` y `/actuator/health`.

Roles disponibles

Rol	Descripción	Acceso
ADMIN	Administrador del sistema	Total (todas las secciones y materias)
COORDINADOR	Coordinador académico	Total (equivalente a ADMIN)
DOCENTE	Docente de aula	Filtrado por secciones y materias asignadas

Evidencia de pruebas funcionales

Caso de prueba	Resultado esperado	Estado
Login con credenciales correctas (admin)	200 OK + accessToken presente	PASS
Login de docente	200 OK + rol = DOCENTE	PASS
Contraseña incorrecta	401 Unauthorized	PASS
Correo inexistente	401 Unauthorized	PASS
Formato de correo inválido	400 Bad Request	PASS
Cuerpo vacío en login	400 Bad Request	PASS
Endpoint protegido sin token	401 Unauthorized	PASS
Endpoint protegido con token inválido	401 Unauthorized	PASS
GET /api/auth/me con token válido	200 OK + datos del usuario	PASS
GET /api/auth/me sin token	401 Unauthorized	PASS
Total		10 / 10

Módulo 2: Control de Acceso por Rol (ContextoUsuario)

Descripción funcional

El **control de acceso basado en rol** es implementado mediante el patrón `ContextoUsuario`: un **record** inmutable que encapsula el alcance del usuario autenticado. Se construye *una vez por request* en `ContextoUsuarioService` y se pasa como parámetro a los servicios que requieren verificar permisos.

Estructura del ContextoUsuario

Listing 3: `ContextoUsuario.java` — record inmutable por request

```
public record ContextoUsuario(  
    Long      usuarioId,  
    RolNombre rol,  
    boolean   esAdmin,      // true para ADMIN y COORDINADOR  
    Set<Long>  seccionIds,   // IDs de secciones asignadas (solo docentes)  
    Set<Integer> materiaIds // IDs de materias asignadas (solo docentes)  
) {  
    // Lanza AccesoDenegadoException (-> 403) si no tiene acceso  
    public void verificarSeccion(Long seccionId) { ... }  
    public void verificarMateria(Integer materiaId) { ... }  
}
```

Flujo de construcción del contexto

1. `JwtAuthenticationFilter` valida el token y carga el `UserDetails`.
2. El controller llama a `ContextoUsuarioService.obtenerContextoActual()`.
3. El servicio lee `SecurityContextHolder`, consulta `usuario_secciones` y `usuario_materias` en BD, y construye el `ContextoUsuario`.
4. Los services de dominio reciben el contexto como parámetro y llaman a `contexto.verificarSeccion` o `contexto.verificarMateria(id)`.

Filtrado de catálogos por rol

El catálogo de materias (GET `/api/catalogos/saberes/materias`) aplica el filtro automáticamente:

- **ADMIN / COORDINADOR**: devuelve las 5 materias del sistema.
- **DOCENTE**: devuelve únicamente las materias en su `materiaIds`.

Evidencia de pruebas funcionales

Caso de prueba	Resultado esperado	Estado
Admin ve catálogo de materias	5 materias	PASS
Docente Keylor ve materias	Solo Matemáticas (1 materia)	PASS
Coordinador ve materias	5 materias (acceso total)	PASS
Catálogo de materias sin token	401 Unauthorized	PASS
Catálogo tipos de saber	3 tipos (sin filtro por rol)	PASS
Catálogo niveles de desempeño	5 niveles	PASS
Ejes de Matemáticas Conceptual	7 ejes	PASS
Ejes de Español Conceptual	7 ejes	PASS
Secciones del docente Keylor	≥ 1 sección (4°A)	PASS
Secciones de la docente María	≥ 2 secciones	PASS
Total		10 / 10

Módulo 3: Evaluaciones por Saber

Descripción funcional

El módulo de **Evaluaciones por Saber** permite que un docente registre la valoración de un estudiante en cada *eje temático* de una materia, para un periodo lectivo específico. La evaluación cubre un *tipo de saber* (Conceptual, Procedimental o Actitudinal) y contiene un *detalle* por cada eje temático seleccionado (escala 1–5).

Endpoints

Método	Ruta	Descripción
POST	/api/evaluaciones-saber	Registra una nueva evaluación
PUT	/api/evaluaciones-saber/{id}	Recalifica (reemplaza detalles)
GET	/api/evaluaciones-saber/{id}	Consulta por ID
GET	/api/evaluaciones-saber/estudiante/{eId}/periodo/{pId}	Lista por estudiante y periodo
GET	/api/evaluaciones-saber/seccion/{sId}/periodo/{pId}	Lista por sección y periodo
GET	/api/evaluaciones-saber/promedios/estudiante/{eId}/periodo/{pId}	Resumen de promedios del estudiante
GET	/api/evaluaciones-saber/seccion/{sId}/periodo/{pId}	Resumen de promedios de la sección

Contrato de la petición

Listing 4: POST /api/evaluaciones-saber — Request body

```
{
  "estudianteId": 31,
  "periodoId": 1,
  "usuarioId": 5,
  "seccionId": 4,
  "materiaId": 1,
  "tipoSaberId": 1,
  "fechaEvaluacion": "2026-04-15", // opcional
  "observacion": "Observacion libre", // opcional
  "detalles": [
    { "ejeTemaaticoId": 22, "valor": 3 },
    { "ejeTemaaticoId": 23, "valor": 4 },
    { "ejeTemaaticoId": 24, "valor": 3 }
    // ... un objeto por eje evaluado
  ]
}
```

Listing 5: POST /api/evaluaciones-saber — Response body (201 Created)

```
{
  "id": 101,
  "estudianteId": 31,
  "estudianteNombreCompleto": "Andrea Mora Vega",
  "periodoId": 1,
  "periodoNombre": "I Trimestre 2026",
  "materiaId": 1,
  "materiaNombre": "Matematicas",
  "tipoSaberId": 1,
  "tipoSaberNombre": "Saber Conceptual",
  "seccionId": 4,
  "seccionNombre": "4 grado A",
  "detalles": [
    { "ejeTemaaticoId": 22, "ejeNombre": "...", "valor": 3 }
  ],
  "createdAt": "2026-04-15T09:00:00"
}
```

Escala de valoración por saber

Valor	Nivel	Descripción
1	Inicial	No evidencia dominio del eje
2	En desarrollo	Dominio parcial, requiere apoyo constante
3	Intermedio	Dominio básico con algunos errores
4	Logrado	Dominio sólido con mínimos errores
5	Avanzado	Dominio pleno y transferencia de conocimiento

Reglas de negocio

- **Restricción de unicidad:** una evaluación por (`estudiante_id`, `usuario_id`, `periodo_id`) — definida a nivel de BD.
- **Validaciones de entrada:** los detalles deben tener al menos 1 entrada (`@Size(min=1)`); el valor debe estar entre 1 y 5 (`@Min(1) @Max(5)`).
- **Recalificación (PUT):** reemplaza todos los detalles existentes usando `orphanRemoval=true`, luego inserta los nuevos. No crea una segunda evaluación.
- **Control de acceso:** el servicio verifica que el docente tenga acceso a la sección y a la materia antes de registrar.

Ejes temáticos por materia

Cada materia tiene **21 ejes temáticos** independientes (7 por tipo de saber):

Materia	Tipo de saber	Ejes	IDs (ejemplo)
Español	Conceptual	7	1–7
	Procedimental	7	8–14
	Actitudinal	7	15–21
Matemáticas	Conceptual	7	22–28
	Procedimental	7	29–35
	Actitudinal	7	36–42
Ciencias, Est. Sociales	Cada uno 3 tipos	7 c/u	(v. V6)

Evidencia de pruebas funcionales

Caso de prueba	Resultado esperado	Estado
Registrar evaluación válida	201 Created + ID generado	PASS
Registrar y recuperar por ID	Mismo ID y detalles coherentes	PASS
Registrar y consultar promedios	200 OK + promediosPorTipoSaber presente	PASS
Listar por estudiante y periodo	Lista con ≥ 1 evaluación	PASS
Registrar sin token	401 Unauthorized	PASS
Registrar con detalles vacíos	400 Bad Request	PASS
Registrar con valor fuera de rango (6)	400 Bad Request	PASS
Buscar por ID inexistente (999999)	404 Not Found	PASS
Total		8 / 8

Módulo 4: Alertas Temáticas

Descripción funcional

El módulo de **Alertas Temáticas** es el núcleo del sistema de alerta temprana. Analiza las evaluaciones registradas para un estudiante (o una sección completa) en un periodo, calcula el promedio por eje temático y genera automáticamente alertas para los ejes cuyo promedio no alcanza el umbral de desempeño esperado.

Endpoints

Método	Ruta	Descripción
POST	/api/alertas-tematicas/generar/estudiante/{eId}/periodo/{pId}	Genera alertas para un estudiante
POST	/api/alertas-tematicas/generar/seccion/{sId}/periodo/{pId}	Genera alertas para toda la sección
GET	/api/alertas-tematicas/estudiante/{eId}/periodo/{pId}	Consulta alertas de un estudiante
GET	/api/alertas-tematicas/seccion/{sId}/periodo/{pId}	Consulta alertas de una sección

Respuesta de alerta

Listing 6: Ejemplo de alerta generada (nivel ALTA)


```
{
  "id": 15,
  "estudianteId": 32,
  "estudianteNombreCompleto": "Luis Perez Mora",
  "periodoId": 1,
  "periodoNombre": "I Trimestre 2026",
  "ejeTemaaticoId": 22,
  "ejeNombre": "Algebra y pensamiento algebraico",
  "ejeClave": "MAT_C_ALGEBRA",
  "materiaId": 1,
  "materiaNombre": "Matematicas",
  "tipoSaberId": 1,
  "tipoSaberNombre": "Saber Conceptual",
  "promedio": 1.43,
  "nivelAlerta": "ALTA",
  "motivo": "Luis Perez Mora presenta promedio 1.43 en Matematicas ...",
  "estado": "ACTIVA",
  "fechaGeneracion": "2026-04-15T09:05:00"
}
```

Reglas de generación de alertas

Niveles de alerta por promedio de eje temático:

Promedio	Nivel	Interpretación
$\leq 2,0$	ALTA	Riesgo crítico. Requiere intervención inmediata.
2,1 – 3,0	MEDIA	Riesgo moderado. Requiere seguimiento.
$> 3,0$	(SIN ALERTA)	Desempeño aceptable. No se genera ni persiste alerta.

Reglas adicionales:

- Las alertas de nivel SIN_ALERTA **no se persisten** en la base de datos.
- Si ya existe una alerta para el mismo (estudiante, periodo, eje, materia), la nueva generación la reemplaza.
- El estado inicial de toda alerta generada es ACTIVA.
- La generación por sección delega internamente en la generación por estudiante para cada alumno matriculado.

Evidencia de pruebas funcionales

Caso de prueba	Resultado esperado	Estado
Valores 1 (promedio ≤ 2) generan alerta	Nivel = ALTA	PASS
Valores 5 (promedio > 3) no generan alertas	Lista vacía (SIN_ALERTA no se persiste)	PASS
Generar y consultar — cantidad coherente	GET retorna misma cantidad que POST	PASS
Generar alertas sin token	401 Unauthorized	PASS
Estudiante sin evaluaciones	Lista vacía (200 OK)	PASS
Total		5 / 5

Módulo 5: Catálogos de Saberes

Descripción funcional

Los catálogos de saberes exponen la información de referencia que el frontend necesita para construir los formularios de evaluación: tipos de saber, materias (filtradas por rol), ejes temáticos por materia y tipo, y niveles de desempeño.

Endpoints

Método	Ruta	Descripción
GET	/api/catalogos/saberes/tipos	Lista los 3 tipos de saber
GET	/api/catalogos/saberes/materias	Lista materias (filtrado por rol del usuario)
GET	/api/catalogos/saberes/ejes	Lista ejes (?materiaId=&tipoSaberId=)
GET	/api/catalogos/saberes/niveles-desempeno	Lista los 5 niveles de desempeño

Datos de referencia

Tipos de saber (3):

ID	Clave	Nombre
1	CONCEPTUAL	Saber Conceptual
2	PROCEDIMENTAL	Saber Procedimental
3	ACTITUDINAL	Saber Actitudinal

Niveles de desempeño (5):

Valor	Nombre	Descripción breve
1	Inicial	No evidencia dominio
2	En desarrollo	Dominio parcial
3	Intermedio	Dominio básico
4	Logrado	Dominio sólido
5	Avanzado	Dominio pleno con transferencia

Materias (5):

ID	Nombre	Ejes totales
1	Español	21 (7 por tipo)
2	Matemáticas	21 (7 por tipo)
3	Ciencias	21 (7 por tipo)
4	Estudios Sociales	21 (7 por tipo)
5	(adicional)	—

Módulo 6: Secciones y Docentes

Descripción funcional

El módulo de secciones gestiona la estructura académica: grupos de estudiantes asignados a un docente en un año lectivo. Expone catálogos de soporte (niveles, centros, docentes) para que el frontend pueda crear nuevas secciones sin depender de datos codificados.

Endpoints

Método	Ruta	Descripción
GET	/api/secciones	Lista secciones (?anioLectivoId=)
GET	/api/secciones/docente/{docenteId}	Secciones asignadas a un docente
POST	/api/secciones	Crea una nueva sección
PUT	/api/secciones/{id}	Actualiza una sección
DELETE	/api/secciones/{id}	Elimina una sección
GET	/api/secciones/catalogos/niveles	Catálogo de niveles educativos
GET	/api/secciones/catalogos/centros	Catálogo de centros educativos
GET	/api/secciones/catalogos/docentes	Catálogo de docentes disponibles

Datos de control de acceso para secciones

La tabla `usuario_secciones` relaciona docentes con sus secciones asignadas. La tabla `usuario_materias` relaciona docentes con sus materias asignadas. Ambas tablas son consultadas por `ContextoUsuarioService` al construir el `ContextoUsuario` de cada request.

Usuario	Rol	Secciones / Materias asignadas
admin@atara.mep.go.cr	ADMIN	Todas (sin restricción)
mgarcia@atara.mep.go.cr	DOCENTE	Secciones 1 y 3; todas las materias
kcortes@atara.mep.go.cr	DOCENTE	Sección 4; solo Matemáticas (id=1)
jperez@atara.mep.go.cr	COORDINADOR	Todas (acceso total)

Módulo 7: Extracción PIAD (OCR)

Descripción funcional

El módulo **PIAD** permite al docente cargar el PDF oficial de la Lista PIAD (Plan Individual de Adecuación), que el sistema procesa mediante OCR para extraer automáticamente la nómina de estudiantes con adecuaciones. Esto elimina la necesidad de registrar manualmente a estos estudiantes.

Endpoint

Método	Ruta	Descripción
POST	/api/piad/extraer	Recibe un PDF multipart, aplica OCR y retorna la lista de estudiantes extraída

Tecnología de procesamiento

1. **Apache PDFBox 3.0.3**: renderiza cada página del PDF a imagen raster a 500 DPI.
2. **Tess4j 5.11.0** (wrapper de Tesseract): aplica OCR en español sobre las imágenes.
3. El texto extraído se parsea con expresiones regulares para identificar nombres, cédulas y tipos de adecuación.

Estado actual: La extracción OCR está implementada y operativa. La *persistencia* de los estudiantes extraídos en la base de datos **no está implementada** en esta versión (trabajo pendiente para V12). El endpoint retorna la lista como respuesta JSON pero no la guarda.

Configuración requerida

- Tesseract debe estar instalado con el paquete de idioma español (**spa**).
- Configurar `piad.tessdata.dir` en `application.properties` o la variable de entorno `TESSDATA_PREFIX`.

Infraestructura de Pruebas Funcionales

Descripción

Las pruebas funcionales validan el sistema completo de extremo a extremo: desde la petición HTTP hasta la base de datos. No usan mocks; ejecutan contra una instancia real de PostgreSQL levantada con **Testcontainers**.

Arquitectura de las pruebas

Listing 7: Estructura de clases de prueba

```
IntegrationTestBase (abstracta)
|- AuthFuncionalTest          (10 tests)
|- ControlAccesoFuncionalTest (10 tests)
|- EvaluacionSaberFuncionalTest (8 tests)
|- AlertaTematicaFuncionalTest (5 tests)
-----
Total: 33 tests
```

Características técnicas

- **Base de datos real:** PostgreSQLContainer con imagen postgres:15-alpine. Se usa el patrón *singleton container*: el contenedor arranca *una vez* para toda la sesión JVM y se detiene al finalizar (shutdown hook automático de Testcontainers).
- **Flyway automático:** Spring ejecuta todas las migraciones V1–V11 al levantar el contexto, garantizando datos de seed consistentes.
- **Sin @Transactional:** con RANDOM_PORT el servidor corre en un hilo separado; los commits son reales. El aislamiento se logra con @AfterEach que llama a limpiarEvaluacionesSaber via JdbcTemplate.
- **RestTemplate con NoOpResponseErrorHandler:** no lanza excepción en respuestas 4xx/5xx, permitiendo verificar el código HTTP directamente.
- **AssertJ:** aserciones fluidas sobre el estado HTTP y el cuerpo JSON.

Resumen de resultados

Clase de prueba	Tests	Pass	Fail	Error
AuthFuncionalTest	10	10	0	0
ControlAccesoFuncionalTest	10	10	0	0
EvaluacionSaberFuncionalTest	8	8	0	0
AlertaTematicaFuncionalTest	5	5	0	0
TOTAL	33	33	0	0

Resultado de la ejecución: BUILD SUCCESS

```
$ ./mvnw test -Dtest="*Funcional*"

[INFO] Tests run: 5 Failures: 0 Errors: 0 Skipped: 0 (AlertaTematica)
[INFO] Tests run: 10 Failures: 0 Errors: 0 Skipped: 0 (Autenticacion)
[INFO] Tests run: 10 Failures: 0 Errors: 0 Skipped: 0 (ControlAcceso)
[INFO] Tests run: 8 Failures: 0 Errors: 0 Skipped: 0
      (EvaluacionSaber)
[INFO] Tests run: 33 Failures: 0 Errors: 0 Skipped: 0

[INFO] BUILD SUCCESS
```


Mapa completo de endpoints de la API

Controller	Método	Ruta
AuthController	POST	/api/auth/login
	POST	/api/auth/refresh
	POST	/api/auth/logout
	GET	/api/auth/me
EstudianteController	POST	/api/estudiantes
	GET	/api/estudiantes
	GET	/api/estudiantes/{id}
	PUT	/api/estudiantes/{id}
EvaluacionSaberController	POST	/api/evaluaciones-saber
	PUT	/api/evaluaciones-saber/{id}
	GET	/api/evaluaciones-saber/{id}
	GET	/api/evaluaciones-saber/estudiante/{eId}/periodo/{pId}
	GET	/api/evaluaciones-saber/seccion/{sId}/periodo/{pId}
	GET	/api/evaluaciones-saber/promedios/estudiante/{eId}/
	GET	/api/evaluaciones-saber/promedios/seccion/{sId}/per
AlertaTematicaController	POST	/api/alertas-tematicas/generar/estudiante/{eId}/per
	POST	/api/alertas-tematicas/generar/seccion/{sId}/periodo
	GET	/api/alertas-tematicas/estudiante/{eId}/periodo/{pId}
	GET	/api/alertas-tematicas/seccion/{sId}/periodo/{pId}
CatalogoSaberController	GET	/api/catalogos/saberes/tipos
	GET	/api/catalogos/saberes/materias
	GET	/api/catalogos/saberes/ejes
	GET	/api/catalogos/saberes/niveles-desempeno
SeccionController	GET	/api/secciones
	GET	/api/secciones/docente/{docenteId}
	POST	/api/secciones
	PUT	/api/secciones/{id}
	DELETE	/api/secciones/{id}
	GET	/api/secciones/catalogos/niveles
	GET	/api/secciones/catalogos/centros
	GET	/api/secciones/catalogos/docentes
AnioLectivoController	POST	/api/anios-lectivos
	GET	/api/anios-lectivos
	GET	/api/anios-lectivos/activo
	GET	/api/anios-lectivos/{id}
	PUT	/api/anios-lectivos/{id}
	PUT	/api/anios-lectivos/{id}/activar

(continúa en la siguiente página)

Controller	Método	Ruta
PeriodoController	POST	/api/periodos
	GET	/api/periodos
	GET	/api/periodos/activo
	PUT	/api/periodos/{id}
	PUT	/api/periodos/{id}/activar
AlertaController	GET	/api/alertas/estudiante/{id}
	GET	/api/alertas/seccion/{id}
PiadController	POST	/api/piad/extraer
ReporteController	GET	/api/reportes/estudiante/{id}
VisualizacionController	GET	/api/visualizaciones/seccion/{id}/distribucion

Conclusiones

- El backend de ATARA implementa **7 módulos funcionales** con una API REST estructurada siguiendo la arquitectura en capas y los principios de separación de responsabilidades.
- El **sistema de control de acceso por rol** garantiza que cada docente opere exclusivamente sobre sus secciones y materias asignadas, sin necesidad de lógica condicional en cada endpoint.
- El módulo de **alertas temáticas** automatiza la detección de riesgo académico: un docente registra evaluaciones y el sistema identifica automáticamente qué ejes temáticos requieren atención.
- Las **33 pruebas funcionales** ejecutan contra una base de datos PostgreSQL real con datos de seed completos, validando flujos de negocio end-to-end con un resultado **100 % exitoso**.
- La **migración incremental con Flyway** garantiza reproducibilidad del esquema: 11 migraciones aplicadas de forma determinista en cualquier entorno (desarrollo, pruebas, producción).